

The Geometry of Compression: PCA, t-SNE, and UMAP

Phase 2: Machine Learning Foundations

Introduction: The Curse of Dimensionality

When dealing with modern datasets, we frequently encounter the Curse of Dimensionality. In high-dimensional spaces (e.g., 100+ features), the mathematical concept of “distance” breaks down. The volume of the space grows exponentially, causing every data point to become nearly equidistant from every other data point. Under these conditions, distance-based algorithms like K-Means become completely blind.

To solve this, we must compress the data into a lower-dimensional space (typically 2D or 3D) while preserving its underlying structure. Below is a rigorous mathematical analysis of the three industry-standard algorithms used to achieve this.

1 Principal Component Analysis (PCA)

The Philosophy: *“Smash the data linearly to maximize variance.”*

PCA is an orthogonal linear transformation. It projects data into a new coordinate system such that the greatest variance by any projection lies on the first coordinate (the first principal component), the second greatest variance on the second coordinate, and so on.

The Algorithm and Proof

Given a dataset $X \in \mathbb{R}^{n \times d}$ where columns are centered ($\mu = 0$):

1. **The Optimization Problem:** We seek a projection vector $w \in \mathbb{R}^d$ of unit length ($w^T w = 1$) that maximizes the variance of the projected data Xw . The variance is given by:

$$\frac{1}{n}(Xw)^T(Xw) = \frac{1}{n}w^T X^T X w = w^T \Sigma w \quad (1)$$

where $\Sigma = \frac{1}{n}X^T X$ is the empirical covariance matrix.

2. **Lagrangian Formulation:** We formulate this as a constrained optimization problem using a Lagrange multiplier λ :

$$\mathcal{L}(w, \lambda) = w^T \Sigma w - \lambda(w^T w - 1) \quad (2)$$

3. **Deriving the Eigenvalue Problem:** To find the maximum, we take the partial derivative of $\mathcal{L}$ with respect to w and set it to zero:

$$\frac{\partial \mathcal{L}}{\partial w} = 2\Sigma w - 2\lambda w = 0 \quad (3)$$

$$\Sigma w = \lambda w \quad (4)$$

This proves that the optimal projection vector w must be an **eigenvector** of the covariance matrix Σ , and the variance it explains is exactly its corresponding **eigenvalue** λ (since $w^T \Sigma w = w^T \lambda w = \lambda$).

4. **Projection:** To reduce to k dimensions, we construct a projection matrix $W_{d \times k}$ containing the top k eigenvectors corresponding to the largest eigenvalues. The lower-dimensional representation is:

$$X_{\text{reduced}} = XW \quad (5)$$

The Verdict

Pros: PCA is lightning fast, highly interpretable, preserves **global structure** (the distance between distant clusters is accurate), and allows for exact inverse transformations ($X \approx X_{\text{reduced}} W^T$).

Cons: It is strictly linear. If your data lies on a non-linear manifold (e.g., a rolled-up “Swiss Roll”), PCA will smash it flat, irreparably mixing the data points.

2 t-SNE (t-Distributed Stochastic Neighbor Embedding)

The Philosophy: *“Forget the big picture; just keep my immediate neighbors close.”*

t-SNE is a non-linear manifold learning technique. It converts Euclidean distances into conditional probabilities that represent similarities.

The Algorithm and Mechanics

1. **High-Dimensional Affinities (Gaussian):** We first define the probability $p_{j|i}$ that point x_i picks x_j as its neighbor, assuming a Gaussian distribution centered at x_i :

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)} \quad (6)$$

The variance σ_i^2 is dynamically determined for each point using binary search to match a user-specified *perplexity* (effectively the target number of nearest neighbors).

2. **Symmetrization:** To simplify the gradient and handle outliers, we compute the joint probability p_{ij} in the high-dimensional space:

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n} \quad (7)$$

3. **Low-Dimensional Affinities (Student’s t-distribution):** In the lower-dimensional space (points y_i), t-SNE models the similarities q_{ij} using a Student’s t-distribution with one degree of freedom (a Cauchy distribution). The heavy tails of this distribution prevent the “Crowding Problem” by allowing points to be placed further apart in low dimensions without heavily penalizing the cost function:

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}} \quad (8)$$

4. **Gradient Descent (The Physics Analogy):** t-SNE minimizes the Kullback-Leibler (KL) Divergence between P and Q . The gradient of the KL divergence with respect to the low-dimensional point y_i is remarkably elegant:

$$\frac{\partial C}{\partial y_i} = 4 \sum_j (p_{ij} - q_{ij})(y_i - y_j)(1 + \|y_i - y_j\|^2)^{-1} \quad (9)$$

This equation behaves like an N-body physics simulation. The vector $(y_i - y_j)$ is a “spring” between points. If $p_{ij} > q_{ij}$ (they are close in High-D but far in Low-D), the spring exerts an *attractive force*. If $p_{ij} < q_{ij}$, it exerts a *repulsive force*.

The Verdict

Pros: Produces distinct, beautiful visual clusters by aggressively separating dissimilar points.

Cons: Extremely slow ($\mathcal{O}(N \log N)$ with Barnes-Hut approximations). It completely **destroys global structure** (the distance between two *different* clusters in a t-SNE plot is meaningless). It cannot be parametrically used to transform unseen test data.

3 UMAP (Uniform Manifold Approximation and Projection)

The Philosophy: “Use algebraic topology to preserve both local neighborhoods AND global structure.”

UMAP leverages fuzzy simplicial sets to build a topological representation of the high-dimensional data, and then optimizes a low-dimensional layout to have as similar a topological structure as possible.

The Algorithm and Mechanics

1. **Local Metric Spaces (ρ_i):** UMAP guarantees that every point is connected to at least its nearest neighbor. It computes the distance to the nearest neighbor for point x_i , denoted as ρ_i . The high-dimensional fuzzy weight between x_i and x_j is:

$$v_{j|i} = \exp \left(-\frac{\max(0, \|x_i - x_j\|_2 - \rho_i)}{\sigma_i} \right) \quad (10)$$

By subtracting ρ_i , UMAP normalizes the notion of “distance” based on the local density of the manifold.

2. **Fuzzy Set Union (Symmetrization):** To form a single, undirected graph (a fuzzy simplicial set), the directed edges must be combined. Instead of simple averaging (like t-SNE), UMAP uses the probabilistic t-conorm for fuzzy set union:

$$v_{ij} = v_{j|i} + v_{i|j} - v_{j|i}v_{i|j} \quad (11)$$

3. **Low-Dimensional Representation:** In the target space, UMAP uses a family of curves inspired by the Student's t-distribution to model low-dimensional similarities w_{ij} :

$$w_{ij} = \frac{1}{1 + a\|y_i - y_j\|_2^{2b}} \quad (12)$$

where a and b are parameters fitted to the user-defined hyperparameter `min_dist` (controlling how tightly points are allowed to pack together).

4. **Cross-Entropy Optimization:** Unlike t-SNE's KL Divergence, UMAP minimizes the binary Cross-Entropy C_E between the high-dimensional fuzzy set V and the low-dimensional fuzzy set W :

$$C_E = \sum_{i \neq j} \left(v_{ij} \log \left(\frac{v_{ij}}{w_{ij}} \right) + (1 - v_{ij}) \log \left(\frac{1 - v_{ij}}{1 - w_{ij}} \right) \right) \quad (13)$$

The inclusion of the $(1 - v_{ij})$ term explicitly forces points that are far apart in high dimensions to push away from each other in low dimensions, which is why UMAP successfully preserves **global structure** much better than t-SNE.

The Verdict

Pros: Significantly faster than t-SNE, creates stunning local clusters, preserves global distances, and allows you to apply a fitted UMAP model to unseen test data using `transform()` by adding points to the existing simplicial complex.

Cons: Requires careful tuning of `n_neighbors` (which balances local vs. global view) and `min_dist`.